

Real-world (a cold interview on your own project)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Model answer for: "Interview me about Project 4."

Basic Version

"Project 4 is called ContentForge. It's for teams that need articles written — someone gives it a topic, and it researches it, writes a draft, and checks the facts before it's considered done.

It has five agents. One of them is a supervisor that decides what happens next — it looks at what's been done so far and sends the work to the right specialist: a researcher, a writer, a fact-checker, and a couple of others depending on the topic.

The decision I'd point to: I used a supervisor that can decide the order for itself, instead of a fixed research-then-write-then-check order. I did that because sometimes the fact-checker finds a problem and the work needs to go back to the writer — a fixed order can't do that, but a supervisor can.

The thing I'd improve: right now, if the fact-checker keeps rejecting the same draft, it could loop back and forth without ever stopping. I'd add a limit — after 3 tries, stop and hand it to a person instead of looping forever."

This covers all four beats plainly, in under two minutes, and volunteers a real limitation without being asked — what a nervous first-timer says when they genuinely know their own project but haven't polished the vocabulary yet.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

"ContentForge solves content production for teams that need researched, fact-checked articles without one person owning the whole pipeline manually.

It's built on Doc11's supervisor pattern with five agents: a supervisor that owns routing, a researcher that gathers sources, a writer that drafts from those sources, a fact-checker that verifies claims against the research, and an editor that does a final pass. The supervisor inspects shared state after each agent runs and decides the next hop dynamically, rather than following a hard-coded sequence.

The decision worth calling out: I chose a supervisor over a fixed sequential pipeline specifically because fact-checking isn't a one-way gate — a failed check needs to route *backward* to the writer with the specific issue attached, and a fixed pipeline has no mechanism for that kind of conditional loop. The trade-off I accepted: a supervisor is harder to reason about statically than a fixed pipeline — you can't just read the code top-to-bottom and know the execution order, you have to trace the supervisor's routing logic, which is a real cost for debuggability that I weighed against the correctness gap a fixed pipeline would have left.

The honest limitation: the writer-fact-checker loop currently has no retry cap, so a genuinely unfixable factual disagreement could cycle indefinitely, burning cost with no forward progress. Given more time, I'd add a max-retry count that escalates to human review instead of looping, which is exactly the kind of guardrail Doc14's debugging lab flags as a standing risk in any agent loop with a conditional back-edge."

Why this answer works: it's this document's Normal-depth rung, done properly — it opens with the user-facing problem, not the tech stack, which is what shapes an interviewer's first impression. It names the pattern *and* the specific requirement that pattern satisfies (backward routing on fact-check failure), which is what "requirements → architecture is matching, not inspiration" (Doc16) actually looks like in speech. It states a real trade-off it accepted, not just a benefit, showing the choice was weighed, not just picked. And it volunteers the limitation with a concrete fix, unprompted.

What the Basic Version misses: the Basic Version's four beats are all correct, but each one stays at the surface level ("a supervisor that can decide," "it could loop back and forth") — the Intermediate Version names the *specific mechanism* underneath each beat (dynamic routing over shared state, a hard trade-off in static reasoning, a named guardrail pattern from Doc14) instead of describing the shape of the idea alone.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Curveball 1 — the challenge, right after the Intermediate answer's "one decision": *"Isn't a full supervisor added complexity for one edge case? A fixed pipeline with a single retry loop around the fact-check step would handle 'send it back to the writer' too, and it's simpler to reason about."*

Model response: "That's fair, and for exactly one back-edge, a fixed pipeline with a single conditional retry loop probably would be enough — I'll concede that directly. The reason I went with a supervisor anyway is that fact-checking wasn't the only place I expected a back-edge to show up. If a future specialist — say, an SEO-compliance check, or a tone-consistency pass — also needs the ability to send work backward to a different stage, a fixed pipeline needs a new hard-coded loop for every one of those, and they start interacting with each other in ways that get hard to follow. A supervisor already generalizes to that without new wiring each time. So the honest framing is: it's a bet on the pipeline growing more conditional paths over time, not a requirement the very first version strictly needed. If I were being graded purely on the requirements as they stood on day one, a fixed retry loop was probably the simpler, equally correct choice — the supervisor is me trading some debuggability now for not having to re-architect later if that bet pays off."

Curveball 2 — the cold project switch: *"Interesting — tell me about a different project instead, one where something went wrong."*

Model response, applying the same four-beat shape on the spot to a different project

(Doc08's RAG pipeline, as an example): "Sure — the RAG project from Doc08. The problem was letting support staff ask questions against a large internal doc set instead of searching manually. The shape: documents get chunked and embedded into a vector store, a retriever pulls the top matches for a query, and a generation step answers using only those matches. The decision worth calling out: I set the retriever to a fixed top-k of 5 chunks, instead of a similarity-score cutoff, because it made latency and cost predictable per query — the trade-off was that on questions where the real answer needed 8 relevant chunks, it silently returned an incomplete answer instead of failing loudly. That's actually the thing that went wrong: it took a user complaint, not a metric, to catch it, because a confidently-worded partial answer doesn't look broken. The fix I added afterward was a similarity-score floor combined with the top-k cap, so it stops early on a strong single match but keeps pulling past 5 when the top results are all weakly similar to each other."

Why this answer works: Curveball 1's response concedes the true part of the challenge immediately instead of defending everything, then reframes the decision as a bet tied to a specific future need rather than a requirement that already existed — which is a more honest and more senior answer than insisting the supervisor was obviously correct from day one. Curveball 2 shows the four-beat structure is a genuine reusable habit, not a script memorized for one project — and it

volunteers a real failure ("it took a user complaint to catch it") without being asked twice, which is exactly the instinct Doc14's debugging material is checking for.

What a weaker answer misses: on Curveball 1, a weaker answer either fully concedes ("yeah, you're right, I over-engineered it") — abandoning a decision that had a real, specific justification the interviewer just hadn't heard yet — or argues without conceding anything, which reads as unable to take feedback. On Curveball 2, a weaker answer either can't produce the four beats for an unprepared project at all, or gives a plain list of features with no problem statement, decision, or limitation — revealing the structure was memorized per-project rather than genuinely internalized.

Which one should you actually give in a real interview? Open with the Basic Version's plain problem statement — that earns the interviewer's attention and trust in the first 15 seconds, regardless of their technical depth. Let your explanation rise naturally into the Intermediate Version's vocabulary and trade-off language as you go. Don't pre-load the Advanced Version's pushback response or a second project before anyone asks — but when a challenge does land, concede what's true first, then defend the specific thing that's still true; and keep the four-beat shape general enough in your head that it works for any project you're asked about, not just the one you rehearsed tonight.